

Funções de perda

Inteligência Artificial Generativa

Tipos básicos de predição e suas respectivas saídas:

- **Regressão univariada:** $y \in \mathbb{R}$
- **Classificação binária:** $y \in \{0, 1\}$
- **Classificação multiclasse:** $y \in \{1, 2, \dots, K\}$

Modelo de redes neurais: família de funções $f[x; \varphi]$ com parâmetros φ que mapeiam entradas para saídas.

Objetivo do treinamento: encontrar φ que produza o “melhor mapeamento” da entrada para a saída.

O que significa “melhor mapeamento”?

Definimos um conjunto de treinamento

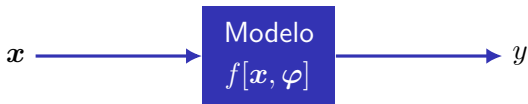
$$\mathcal{D} = \{(\mathbf{x}_i, y_i)\}_{i=1}^I.$$

Função de perda $L[\varphi]$:

- Mede o desajuste entre $f[\mathbf{x}_i; \varphi]$ e y_i .
- Retorna um número escalar.
- Buscamos minimizar $L[\varphi]$.

Como construir funções de perda?

Seja um modelo $f[x, \varphi]$ com parâmetros φ que computa uma saída y a partir da entrada x .



Perspectiva probabilística: o modelo define uma distribuição de probabilidade condicional

$$\Pr(y \mid \mathbf{x})$$

sobre as possíveis saídas y dado o valor de entrada $\mathbf{x}$.



Lembrete:

- Distribuição condicional de x dado θ :

$$\Pr(x \mid \theta) \leftarrow \text{vista como função de } x$$

- Precisa somar/integrar para 1 em relação a x
- Verossimilhança de x dado θ .

$$\Pr(x \mid \theta) \leftarrow \text{vista como função de } \theta$$

- Não precisa somar/integrar para 1 em relação a θ .

A função de perda incentiva que cada saída de treinamento y_i tenha alta verossimilhança sob a distribuição

$$\Pr(y_i \mid \mathbf{x}_i)$$

calculada a partir da entrada correspondente $\mathbf{x}_i$.

Funções de perda

- **Tarefa de regressão:**

- **Objetivo:** prever uma saída contínua $y \in \mathbb{R}$ a partir da entrada x , com base em dados de treinamento

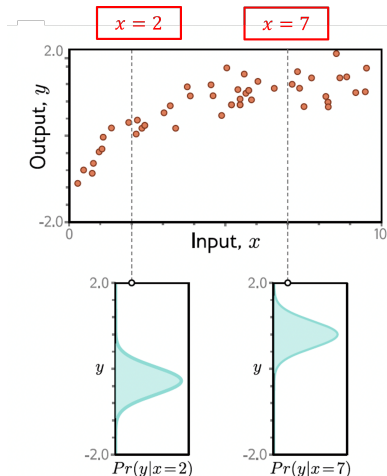
$$\mathcal{D} = \{(\mathbf{x}_i, y_i)\}_{i=1}^I.$$

- Dado um valor de entrada x , o modelo prediz uma distribuição

$$\Pr(y \mid x)$$

sobre as saídas y .

- A função de perda busca **maximizar** a verossimilhança dos valores observados y_i sob a distribuição predita para a entrada correspondente x_i .



Simon J. D. Prince. Understanding Deep Learning, The MIT Press, 2023.

Funções de perda

- **Tarefa de classificação (K classes):**

- **Objetivo:** prever uma saída discreta $y \in \{1, \dots, K\}$ a partir da entrada $\mathbf{x}$, com base em dados de treinamento

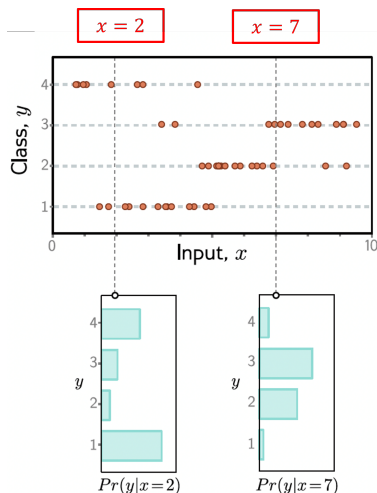
$$\mathcal{D} = \{(\mathbf{x}_i, y_i)\}_{i=1}^I.$$

- Dado um valor de entrada $\mathbf{x}$, o modelo prediz uma distribuição discreta

$$\Pr(y \mid \mathbf{x})$$

(categórica) sobre as saídas y .

- A função de perda busca **maximizar** a verossimilhança dos valores observados y_i sob a distribuição predita para a entrada correspondente $\mathbf{x}_i$.



Simon J. D. Prince. Understanding Deep Learning, The MIT Press, 2023.

Como adaptar um modelo $f[x, \varphi]$ para calcular uma distribuição de probabilidade?

- **Passos:**

- 1 Escolhemos uma distribuição paramétrica $\Pr(y \mid \theta)$ definida no domínio das saídas y .
- 2 Usamos a rede neural para calcular um ou mais dos parâmetros θ dessa distribuição.

Exemplo:

Suponha que $y \in \mathbb{R}$.

- 1 Possível escolha de distribuição: normal univariada (definida sobre $\mathbb{R}$).
- 2 Parâmetros da distribuição normal univariada:

$$\theta = \{\text{média, variância}\} = \{\mu, \sigma^2\}.$$

O modelo de aprendizado de máquina pode prever a média, enquanto a variância pode ser tratada como uma constante desconhecida.

- **Suposição:** os dados são independentes e identicamente distribuídos (i.i.d.)
 - A forma de $\Pr(y_i | \mathbf{x}_i)$ é a mesma para todos os dados.
 - As distribuições condicionais $\Pr(y_i | \mathbf{x}_i)$ das saídas dadas as entradas são independentes.
- Dessa forma, a distribuição conjunta das saídas dadas as entradas é:

$$\Pr(\mathbf{y} | \mathbf{x}) = \Pr(y_1, y_2, \dots, y_I | x_1, x_2, \dots, x_I) = \prod_{i=1}^I \Pr(y_i | x_i).$$

Critério de máxima verossimilhança

- Para cada entrada $\mathbf{x}_i$ de $\mathcal{D}$, o modelo calcula diferentes parâmetros de distribuição

$$\boldsymbol{\theta}_i = f[\mathbf{x}_i, \boldsymbol{\varphi}].$$

- Cada saída y_i de $\mathcal{D}$ deve ter **alta verossimilhança** sob sua distribuição correspondente $\Pr(y_i \mid \boldsymbol{\theta}_i)$.
- Escolhemos os parâmetros do modelo $\boldsymbol{\varphi}$ de forma a maximizar a verossimilhança combinada em todos os I exemplos de treinamento:

$$\hat{\boldsymbol{\varphi}} = \arg \max_{\boldsymbol{\varphi}} \prod_{i=1}^I \Pr(y_i \mid \mathbf{x}_i)$$

$$= \arg \max_{\boldsymbol{\varphi}} \prod_{i=1}^I \Pr(y_i \mid \boldsymbol{\theta}_i)$$

$$= \arg \max_{\boldsymbol{\varphi}} \prod_{i=1}^I \Pr(y_i \mid f[\mathbf{x}_i, \boldsymbol{\varphi}]) \quad \leftarrow \text{critério de máxima verossimilhança}$$

Problema: critério de máxima verossimilhança não é prático

- Cada termo $\Pr(y_i \mid f[\mathbf{x}_i, \varphi])$ pode ser muito pequeno.
- O produto de muitos termos pode ser minúsculo.
- Problemas com aritmética de precisão finita.

Solução: Maximizar o logaritmo da verossimilhança

$$\begin{aligned}\hat{\varphi} &= \arg \max_{\varphi} \prod_{i=1}^I \Pr(y_i \mid f[\mathbf{x}_i, \varphi]) \\ &= \arg \max_{\varphi} \left[\log \left(\prod_{i=1}^I \Pr(y_i \mid f[\mathbf{x}_i, \varphi]) \right) \right] \\ &= \arg \max_{\varphi} \left[\sum_{i=1}^I \log (\Pr(y_i \mid f[\mathbf{x}_i, \varphi])) \right]\end{aligned}$$

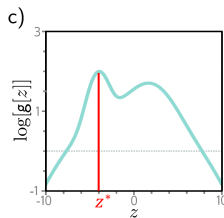
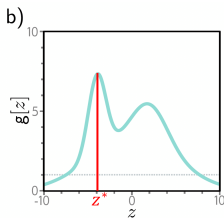
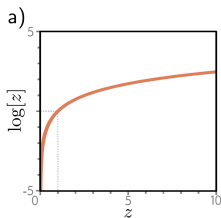
- Soma de termos é mais fácil de representar do que produto.

Por que usar o logaritmo é válido?

Critério de máxima verossimilhança

Por que usar o logaritmo é válido?

- O logaritmo é uma função monotonicamente crescente.
 - Se $z > z'$, então $\log(z) > \log(z')$.
- Melhorar a log-verossimilhança implica melhorar a verossimilhança original.
- Os máximos dos dois critérios ocorrem no mesmo ponto.



$$\begin{aligned} \operatorname{argmax}_z [g(z)] \\ &= \\ \operatorname{argmax}_z [\log(g(z))] \\ &= \\ z^* \end{aligned}$$

Simon J. D. Prince. Understanding Deep Learning, The MIT Press, 2023.

Minimização da Log-Verossimilhança Negativa

- O treinamento de modelos é formulado como um problema de **minimização** de uma função de perda.
- Multiplicamos a log-verossimilhança por **-1** para convertê-la em uma função de perda.
- **Critério da log-verossimilhança negativa:**

$$\hat{\varphi} = \arg \min_{\varphi} \underbrace{\left[- \sum_{i=1}^I \log \left(\Pr(y_i \mid \mathbf{f}[\mathbf{x}_i, \varphi]) \right) \right]}_{L(\varphi)}$$
$$= \arg \min_{\varphi} [L(\varphi)]$$

- $L(\varphi)$: função de perda [*negative log-likelihood*, **NLL**]

Inferência: Predição a partir da distribuição

- A rede não prevê diretamente $\mathbf{y}$, mas **define uma distribuição de probabilidade** sobre $\mathbf{y}$.
- Na inferência, geralmente estamos interessados em uma **estimativa pontual**:

$$\hat{\mathbf{y}} = \arg \max_{\mathbf{y}} \Pr(\mathbf{y} \mid f[\mathbf{x}, \hat{\varphi}])$$

- $f[\mathbf{x}, \hat{\varphi}]$: modelo treinado.
- Geralmente podemos encontrar uma expressão para $\hat{\mathbf{y}}$ em termos dos parâmetros θ da distribuição que são estimados pelo modelo.
 - Exemplo: em uma distribuição normal univariada, o máximo ocorre na média μ .

Receita para construir funções de perda I

Considere um conjunto de dados de treinamento $\mathcal{D} = \{(\mathbf{x}_i, y_i)\}_{i=1}^I$. A receita para construir funções de perda utilizando o método da estimação por máxima verossimilhança é:

- 1 Escolher uma distribuição de probabilidade $\Pr(\mathbf{y} \mid \boldsymbol{\theta})$ adequada ao tipo de predição.
- 2 Definir o modelo $\mathbf{f}[\mathbf{x}, \boldsymbol{\varphi}]$ para prever os parâmetros $\boldsymbol{\theta}$ da distribuição escolhida:

$$\boldsymbol{\theta} = \mathbf{f}[\mathbf{x}, \boldsymbol{\varphi}], \quad \Pr(\mathbf{y} \mid \boldsymbol{\theta}) = \Pr(\mathbf{y} \mid \mathbf{f}[\mathbf{x}, \boldsymbol{\varphi}])$$

- 3 Ajustar os parâmetros do modelo $\boldsymbol{\varphi}$ minimizando a log-verossimilhança negativa sobre $\mathcal{D}$:

$$\hat{\boldsymbol{\varphi}} = \arg \min_{\boldsymbol{\varphi}} [L(\boldsymbol{\varphi})] = \arg \min_{\boldsymbol{\varphi}} \left[- \sum_{i=1}^I \log (\Pr(y_i \mid \mathbf{f}[\mathbf{x}_i, \boldsymbol{\varphi}])) \right]$$

Receita para construir funções de perda II

4 Inferência: para novo exemplo $\mathbf{x}$

- Retornar a distribuição completa:

$$\Pr(\mathbf{y} \mid f[\mathbf{x}, \hat{\varphi}])$$

ou

- Retornar o valor onde a distribuição é maximizada:

$$\hat{\mathbf{y}} = \arg \max_{\mathbf{y}} \Pr(\mathbf{y} \mid f[\mathbf{x}, \hat{\varphi}])$$

Exemplo 1: Regressão Univariada

Objetivo: prever uma saída escalar $y \in \mathbb{R}$ a partir da entrada x .

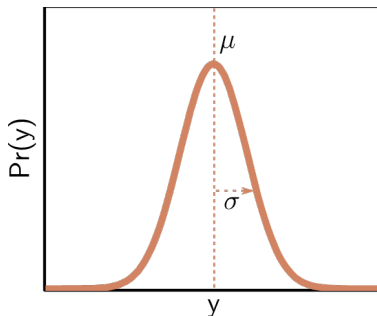
Modelo: $f[x, \varphi]$ com parâmetros φ .

Passo 1: Escolhemos uma distribuição normal univariada sobre y :

$$\Pr(y \mid \mu, \sigma^2) = \frac{1}{\sqrt{2\pi\sigma^2}} \exp\left(-\frac{(y - \mu)^2}{2\sigma^2}\right)$$

- Parâmetros da distribuição:
 - média μ
 - variância σ^2

Distribuição normal univariada



Simon J. D. Prince. Understanding Deep Learning, The MIT Press, 2023.

Exemplo 1: Regressão Univariada

Passo 2: O modelo $f[\mathbf{x}, \varphi]$ deve estimar um ou mais parâmetros da distribuição escolhida.

- Verossimilhança:

$$\Pr(y \mid \underbrace{f[\mathbf{x}, \varphi]}_{\substack{\text{modelo prediz} \\ \text{a média } \mu}}, \sigma^2) = \frac{1}{\sqrt{2\pi\sigma^2}} \exp\left(-\frac{(y - f[\mathbf{x}, \varphi])^2}{2\sigma^2}\right)$$

Passo 3: Buscam-se os parâmetros φ que tornam o conjunto de dados de treinamento $\mathcal{D} = \{(\mathbf{x}_i, y_i)\}_{i=1}^I$ mais provável sob a distribuição escolhida. Para isso, utiliza-se a função de perda baseada na NLL:

$$\begin{aligned} L(\varphi) &= - \sum_{i=1}^I \log\left(\Pr(y_i \mid f[\mathbf{x}_i, \varphi], \sigma^2)\right) \\ &= - \sum_{i=1}^I \log\left(\frac{1}{\sqrt{2\pi\sigma^2}} \exp\left(-\frac{(y_i - f[\mathbf{x}_i, \varphi])^2}{2\sigma^2}\right)\right) \end{aligned}$$

Exemplo 1: Regressão Univariada

Derivando a função de perda de mínimos quadrados

$$\hat{\varphi} = \arg \min_{\varphi} [L(\varphi)]$$

$$= \arg \min_{\varphi} \left[- \sum_{i=1}^I \log \left(\frac{1}{\sqrt{2\pi}\sigma^2} \exp \left(- \frac{(y_i - f[\mathbf{x}_i, \varphi])^2}{2\sigma^2} \right) \right) \right]$$

$$= \arg \min_{\varphi} \left[- \sum_{i=1}^I \left(\log \left(\frac{1}{\sqrt{2\pi}\sigma^2} \right) - \frac{(y_i - f[\mathbf{x}_i, \varphi])^2}{2\sigma^2} \right) \right]$$

O termo $\log \left(\frac{1}{\sqrt{2\pi}\sigma^2} \right)$ independe de φ

$$= \arg \min_{\varphi} \left[- \sum_{i=1}^I \left(- \frac{1}{2\sigma^2} (y_i - f[\mathbf{x}_i, \varphi])^2 \right) \right]$$

O termo $\frac{1}{2\sigma^2}$ independe de φ

$$= \arg \min_{\varphi} \left[\sum_{i=1}^I (y_i - f[\mathbf{x}_i, \varphi])^2 \right]$$

← Mínimos quadrados

Suposições:

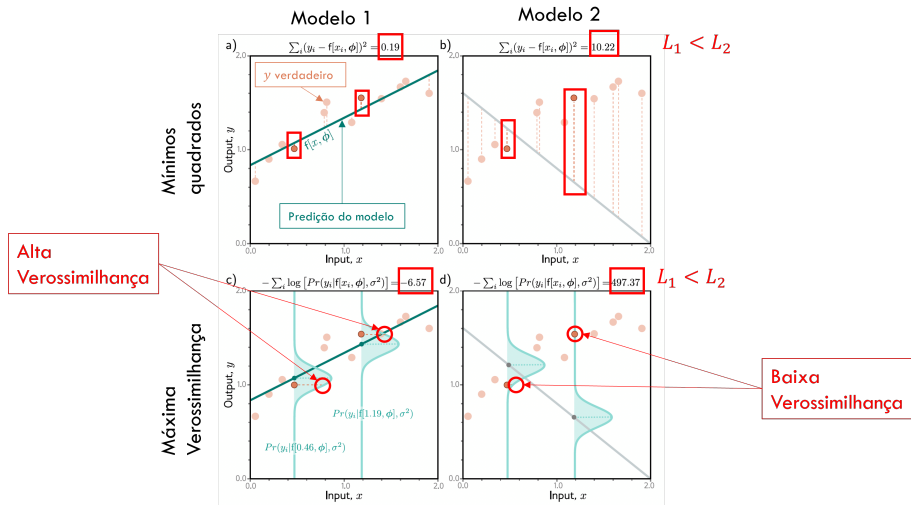
- 1 Predições independentes;
- 2 Amostragem de uma distribuição normal com média $\mu = f[\mathbf{x}_i, \varphi]$.

O que podemos observar?



Exemplo 1: Regressão Univariada

O que podemos observar?



Adaptado de Simon J. D. Prince. Understanding Deep Learning, The MIT Press, 2023.

Exemplo 1: Regressão Univariada

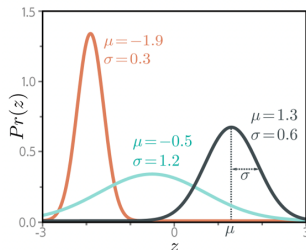
Inferência: Estimativa Pontual

- Modelo prediz a média da distribuição normal sobre y : $\mu = f[\mathbf{x}_i, \hat{\varphi}]$.
- Geralmente queremos uma única e melhor estimativa pontual $\hat{y}$, então tomamos o máximo da distribuição predita:

$$\hat{y} = \arg \max_y \Pr(y \mid f[\mathbf{x}, \hat{\varphi}], \sigma^2)$$

- Para a distribuição normal: o máximo ocorre em μ .

$$\hat{y} = f[\mathbf{x}_i, \hat{\varphi}] = \mu$$



Exemplo 1: Regressão Univariada

Estimação da variância

- Suposição inicial: rede prediz a média de uma distribuição normal.
- Função de perda que vimos não depende da variância σ^2 .
- É possível aprender a variância σ^2 junto com os parâmetros do modelo.
- **Regressão homoscedástica:**

$$L(\varphi, \sigma^2) = - \sum_{i=1}^I \log \left(\frac{1}{\sqrt{2\pi\sigma^2}} \exp \left(-\frac{(y_i - f[\mathbf{x}_i, \varphi])^2}{2\sigma^2} \right) \right)$$

$$\hat{\varphi}, \hat{\sigma}^2 = \arg \min_{\varphi, \sigma^2} [L(\varphi, \sigma^2)]$$

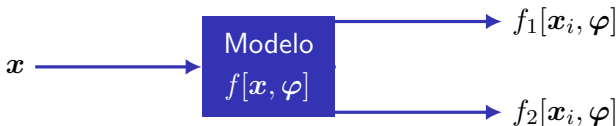
Durante a inferência:

- Modelo prediz a média (melhor predição);
- Variância aprendida indica incerteza da predição.

Exemplo 1: Regressão Univariada

Regressão heteroscedástica

- Modelo anterior assume variância constante (homoscedástico).
- Em muitos casos, a variância varia com os dados (heteroscedástico).
- **Modelagem:** rede neural com duas saídas
 - 1 $f_1[x_i, \varphi]$: predição da média
 - 2 $f_2[x_i, \varphi]$: predição da variância



Qual é o problema com essa modelagem?

Exemplo 1: Regressão Univariada

Regressão heteroscedástica

• Problema

- Variância deve ser sempre positiva.
- Rede neural pode produzir valores negativos.

• Solução

- Aplicar função que gera saída positiva. Dessa forma temos:

$$\mu = f_1[\mathbf{x}, \varphi]$$

$$\sigma^2 = (f_2[\mathbf{x}, \varphi])^2$$

• Função de perda

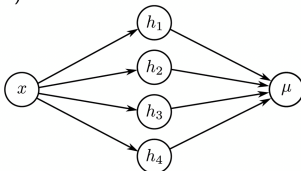
$$L(\varphi) = - \sum_{i=1}^I \log \left(\frac{1}{\sqrt{2\pi (f_2[\mathbf{x}_i, \varphi])^2}} \exp \left(- \frac{(y_i - f_1[\mathbf{x}_i, \varphi])^2}{2(f_2[\mathbf{x}_i, \varphi])^2} \right) \right)$$

$$\hat{\varphi} = \arg \min_{\varphi} [L(\varphi)]$$

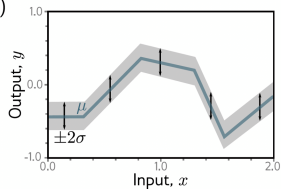
Exemplo 1: Regressão Univariada

Regressão homoscedástica: a) e b)

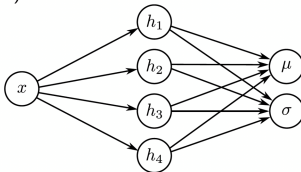
a)



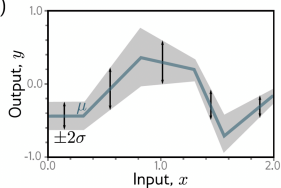
b)



c)



d)



Regressão heteroscedástica: c) e d)

Adaptado de Simon J. D. Prince. Understanding Deep Learning, The MIT Press, 2023.

Exemplo 2: Classificação Binária

Objetivo: atribuir um rótulo discreto $y \in \{0, 1\}$ para cada entrada x .

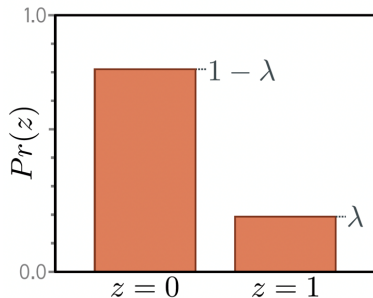
Modelo: $f[x, \varphi]$ com parâmetros φ .

Passo 1: Escolhemos uma distribuição de Bernoulli sobre y :

$$\Pr(y \mid \lambda) = (1 - \lambda)^{1-y} \lambda^y$$

- Parâmetros da distribuição:
 - probabilidade $\lambda \in [0, 1]$

Distribuição de Bernoulli



$$\Pr(z = 1) = \lambda$$

$$\Pr(z = 0) = 1 - \lambda$$

Simon J. D. Prince. Understanding Deep Learning, The MIT Press, 2023.

Exemplo 2: Classificação Binária

O modelo $f[\mathbf{x}, \varphi]$ deve estimar o parâmetro da distribuição escolhida.

Problema

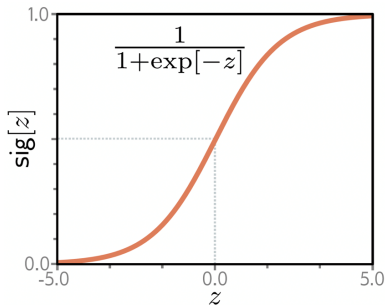
- Restrição: $\lambda \in [0, 1]$.

Solução

- Aplicar função logística sigmóide.

$$\text{sig}[z] = \frac{1}{1 + \exp[-z]}, \quad \text{sig} : \mathbb{R} \rightarrow [0, 1]$$

Função logística sigmóide



Simon J. D. Prince. Understanding Deep Learning, The MIT Press, 2023.

Exemplo 2: Classificação Binária

Passo 2: O modelo $f[\mathbf{x}, \varphi]$ deve estimar o parâmetro da distribuição escolhida.

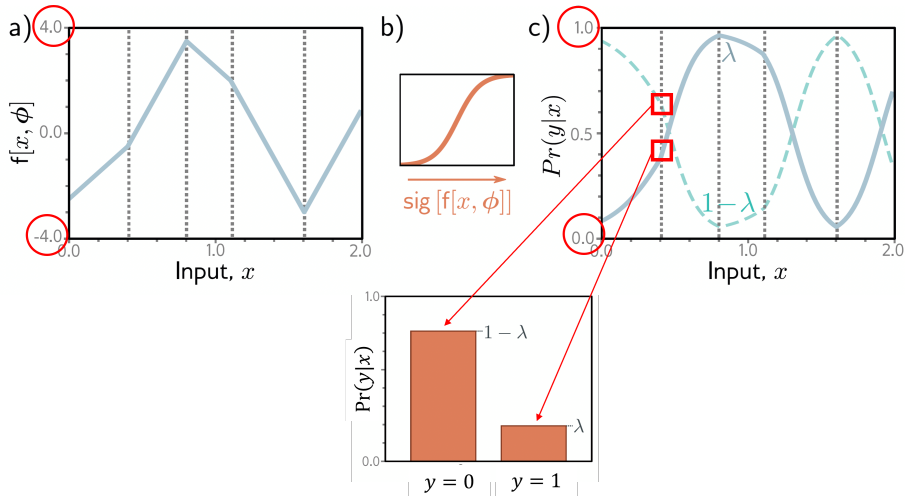
- Verossimilhança:

$$\Pr(y \mid \underbrace{\text{sig}[f[\mathbf{x}, \varphi]]}_{\substack{\text{modelo prediz} \\ \text{a probabilidade } \lambda}}) = (1 - \text{sig}[f[\mathbf{x}, \varphi]])^{1-y} (\text{sig}[f[\mathbf{x}, \varphi]])^y$$

Passo 3: Buscam-se os parâmetros φ que tornam o conjunto de dados de treinamento $\mathcal{D} = \{(\mathbf{x}_i, y_i)\}_{i=1}^I$ mais provável sob a distribuição escolhida. Para isso utiliza-se a função de perda baseada na NLL:

$$\begin{aligned} L(\varphi) &= - \sum_{i=1}^I \log[\Pr(y_i \mid \text{sig}[f[\mathbf{x}_i, \varphi]])] \\ &= - \sum_{i=1}^I \log \left[(1 - \text{sig}[f[\mathbf{x}_i, \varphi]])^{1-y_i} (\text{sig}[f[\mathbf{x}_i, \varphi]])^{y_i} \right] \\ &= - \sum_{i=1}^I [(1 - y_i) \log(1 - \text{sig}[f[\mathbf{x}_i, \varphi]]) + y_i \log(\text{sig}[f[\mathbf{x}_i, \varphi]])] \end{aligned} \quad \leftarrow \text{Entropia cruzada binária}$$

Exemplo 2: Classificação Binária



Adaptado de Simon J. D. Prince. Understanding Deep Learning, The MIT Press, 2023.

Exemplo 2: Classificação Binária

Inferência: Estimativa Pontual

- O modelo prediz o parâmetro da distribuição Bernoulli sobre y :

$$\lambda = \text{sig}[f[\mathbf{x}_i, \hat{\boldsymbol{\varphi}}]].$$

- Desejamos uma única e melhor estimativa pontual $\hat{y}$:

$$\hat{y} = \arg \max_{y \in \{0,1\}} \Pr(y \mid \text{sig}[f[\mathbf{x}, \hat{\boldsymbol{\varphi}}]])$$

- Para a distribuição de Bernoulli:

- Se $\lambda > 0.5 \Rightarrow \hat{y} = 1$.
- Se $\lambda \leq 0.5 \Rightarrow \hat{y} = 0$.

Exemplo 2: Classificação Binária

- Distribuição de Bernoulli
 - Forma da distribuição:

$$\Pr(y \mid \lambda) = (1 - \lambda)^{1-y} \lambda^y$$

- Caso $y = 1$:

$$\Pr(y = 1 \mid \lambda) = (1 - \lambda)^{1-1} \lambda^1 = (1 - \lambda)^0 \lambda = \boxed{\lambda}$$

- Caso $y = 0$:

$$\Pr(y = 0 \mid \lambda) = (1 - \lambda)^{1-0} \lambda^0 = (1 - \lambda)^1 \lambda^0 = \boxed{(1 - \lambda)}$$

- Regra de decisão:

$$\lambda > 0.5 \Rightarrow \text{Pr}(y = 1 \mid \lambda) > \text{Pr}(y = 0 \mid \lambda)$$

$$\Rightarrow \hat{y} = \arg \max_{y \in \{0,1\}} \Pr(y \mid \lambda) = 1$$

$$\lambda \leq 0.5 \Rightarrow \text{Pr}(y = 0 \mid \lambda) \geq \text{Pr}(y = 1 \mid \lambda)$$

$$\Rightarrow \hat{y} = \arg \max_{y \in \{0,1\}} \Pr(y \mid \lambda) = 0$$

Exemplo 3: Classificação Multiclasse

Objetivo: atribuir um rótulo discreto $y \in \{1, 2, \dots, K\}$, com $K > 2$, para cada entrada x .

Modelo: $f[x, \varphi]$ com parâmetros φ .

Passo 1. Escolhemos uma distribuição categórica sobre y :

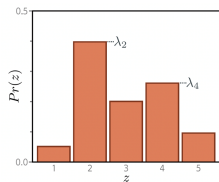
$$\Pr(y = k) = \lambda_k, \quad k = 1, \dots, K$$

- Parâmetros da distribuição:
 - probabilidades $\lambda_1, \dots, \lambda_K$

$$\lambda_k \in [0, 1], \quad \forall k$$

$$\sum_{k=1}^K \lambda_k = 1$$

Distribuição categórica



$$\Pr(z = 1) = \lambda_1$$

⋮

$$\Pr(z = 5) = \lambda_5$$

Simon J. D. Prince. Understanding Deep Learning, The MIT Press, 2023.

Exemplo 3: Classificação Multiclasse

O modelo $f[\mathbf{x}, \boldsymbol{\varphi}]$ deve estimar os K parâmetros da distribuição escolhida.

Problema

- Restrições

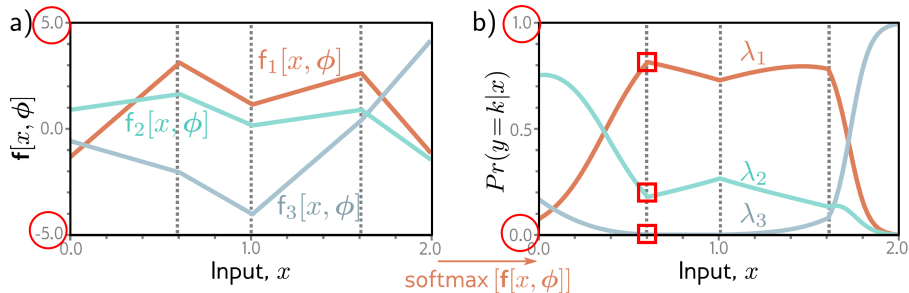
$$\lambda_k \geq 0, \quad \sum_{k=1}^K \lambda_k = 1$$

Solução

- Aplicar função softmax às K saídas da rede.
- k -ésima saída da softmax:

$$\text{softmax}_k[\mathbf{z}] = \frac{\exp[z_k]}{\sum_{k'=1}^K \exp[z_{k']}]$$

Exemplo 3: Classificação Multiclasse



Simon J. D. Prince. Understanding Deep Learning, The MIT Press, 2023.

Exemplo 3: Classificação Multiclasse

Passo 2. O modelo $f[\mathbf{x}, \varphi]$ deve estimar os parâmetros da distribuição escolhida.

- Verossimilhança:

$$\Pr(y = k \mid \mathbf{x}) = \text{softmax}_k[f[\mathbf{x}, \varphi]]$$

Passo 3. Buscam-se os parâmetros φ que tornam o conjunto de dados de treinamento $\mathcal{D} = \{(\mathbf{x}_i, y_i)\}_{i=1}^I$ mais provável sob a distribuição escolhida.

- Função de perda: log-verossimilhança negativa (NLL)

$$\begin{aligned} L(\varphi) &= - \sum_{i=1}^I \log[\text{softmax}_{y_i}[f[\mathbf{x}_i, \varphi]]] \\ &= - \underbrace{\sum_{i=1}^I \left(f_{y_i}[\mathbf{x}_i, \varphi] - \log \left[\sum_{k'=1}^K \exp(f_{k'}[\mathbf{x}_i, \varphi]) \right] \right)}_{\text{Entropia cruzada multiclasse}} \end{aligned}$$

- $f_k[\mathbf{x}, \varphi]$: k -ésima saída da rede neural

Exemplo 3: Classificação Multiclasse

Inferência

- O modelo prediz os parâmetros da distribuição categórica sobre y .
- Estimativa pontual ($\hat{y}$):

$$\hat{y} = \arg \max_k \Pr (y = k \mid f[\mathbf{x}, \hat{\varphi}])$$

- $\hat{y}$ corresponde ao índice k cujo valor de softmax é o maior.
- Isto é, o modelo retorna $\hat{y}$ como a classe com maior valor de softmax (a classe mais provável).

Predição de outros tipos de dados

- Escolher distribuição adequada ao domínio dos dados (a escolha da distribuição depende do tipo de variável-alvo y).
- Aplicar a mesma “receita”.

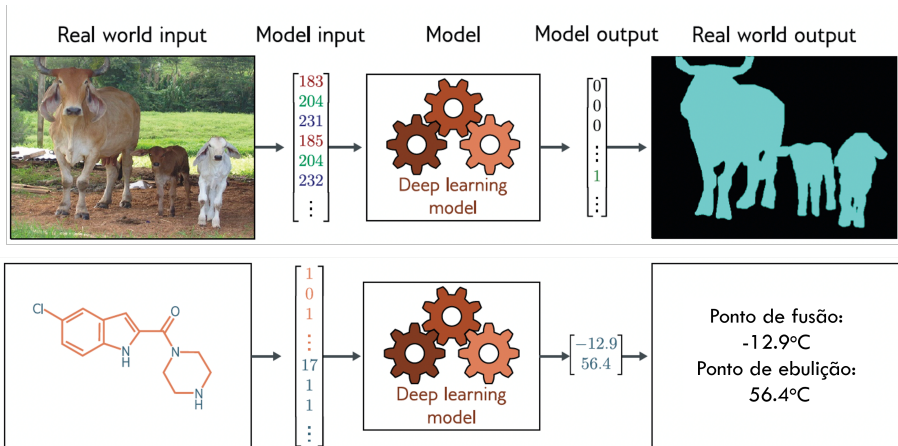
Data Type	Domain	Distribution	Use
univariate, continuous, unbounded	$y \in \mathbb{R}$	univariate normal	regression
univariate, continuous, unbounded	$y \in \mathbb{R}$	Laplace or t-distribution	robust regression
univariate, continuous, unbounded	$y \in \mathbb{R}$	mixture of Gaussians	multimodal regression
univariate, continuous, bounded below	$y \in \mathbb{R}^+$	exponential or gamma	predicting magnitude
univariate, continuous, bounded	$y \in [0, 1]$	beta	predicting proportions
multivariate, continuous, unbounded	$\mathbf{y} \in \mathbb{R}^K$	multivariate normal	multivariate regression
univariate, continuous, circular	$y \in (-\pi, \pi]$	von Mises	predicting direction
univariate, discrete, binary	$y \in \{0, 1\}$	Bernoulli	binary classification
univariate, discrete, bounded	$y \in \{1, 2, \dots, K\}$	categorical	multiclass classification
univariate, discrete, bounded below	$y \in [0, 1, 2, 3, \dots]$	Poisson	predicting event counts
multivariate, discrete, permutation	$\mathbf{y} \in \text{Perm}[1, 2, \dots, K]$	Plackett-Luce	ranking

Figure 5.11 Distributions for loss functions for different prediction types.

Simon J. D. Prince. Understanding Deep Learning, The MIT Press, 2023.

Múltiplas saídas

Muitas vezes desejamos prever um vetor $\mathbf{y} = (y_1, \dots, y_D)$.



Simon J. D. Prince. Understanding Deep Learning, The MIT Press, 2023.

- Em teoria: possível modelar uma distribuição multivariada.
- Na prática: assume-se **independência** entre saídas $y_d \in \mathbf{y}$.
 - Fatoração da probabilidade conjunta:

$$\Pr(\mathbf{y} \mid f[\mathbf{x}, \boldsymbol{\varphi}]) = \prod_{d=1}^D \Pr(y_d \mid f_d[\mathbf{x}, \boldsymbol{\varphi}])$$

- $f_d[\mathbf{x}, \boldsymbol{\varphi}]$: fornece os parâmetros da distribuição de cada saída y_d
- Exemplos:
 - $y_d \in \mathbb{R}$: distribuição normal univariada, $f_d[\mathbf{x}, \boldsymbol{\varphi}]$ prediz a média.
 - $y_d \in \{1, 2, \dots, K\}$: distribuição categórica, $f_d[\mathbf{x}, \boldsymbol{\varphi}]$ prediz as probabilidades.

- Função de perda: log-verossimilhança negativa

$$\begin{aligned} L(\boldsymbol{\varphi}) &= - \sum_{i=1}^I \log[\Pr(\mathbf{y}_i \mid f[\mathbf{x}_i, \boldsymbol{\varphi}])] \\ &= - \sum_{i=1}^I \log \left[\prod_{d=1}^D \Pr(y_{id} \mid f_d[\mathbf{x}_i, \boldsymbol{\varphi}]) \right] \\ &= - \sum_{i=1}^I \sum_{d=1}^D \log[\Pr(y_{id} \mid f_d[\mathbf{x}_i, \boldsymbol{\varphi}])] \end{aligned}$$

- y_{id} : d -ésima saída relativa à i -ésima amostra do conjunto de treinamento
- Predição de múltiplos tipos de variáveis ao mesmo tempo.
 - Suposição de independência (novamente).
 - Verossimilhança conjunta: produto das verossimilhanças individuais.
 - Função de perda: soma das log-verossimilhanças negativas.

- **Perspectiva probabilística:** redes neurais vistas como modelos que predizem parâmetros θ de distribuições $\Pr(y \mid \theta)$.
- **Funções de perda:** construídas a partir da log-verossimilhança negativa (equivalente à entropia cruzada).
- **Critério:** escolher os parâmetros φ da rede neural que maximizam a verossimilhança dos dados.
 - Equivalente a minimizar a log-verossimilhança negativa.
- **Extensões**
 - Predição de diferentes tipos de dados: escolher distribuição adequada.
 - Múltiplas saídas: tratar variáveis como independentes.

- Simon J. D. Prince. *Understanding Deep Learning*. The MIT Press, 2023. [Capítulo 5]
 - Disponível em: <https://udlbook.github.io/udlbook/>

Slides construídos com base no livro acima e em materiais públicos disponibilizados pelo autor.